

IIR Assistive Navigation Device



University
of Glasgow

Adeem Azad

Introduction

Navigating unfamiliar places is one of many serious issues visually impaired people face. Even though conventional aids like guide dogs and canes have been around for decades, they only offer limited information about the user's local environment. To bridge the gap, this project leverages ultrasonic sensors and signal processing techniques to enhance spatial awareness and mobility.

The project is built around an ultrasonic sensor that measures distances with high precision. However, the raw sensor data is often filled with noise and inconsistencies. To address this, the project incorporates an Infinite Impulse Response (IIR) filter, which processes the sensor output to eliminate noise and produce stable, reliable measurements. By refining the real-time data, the device ensures that users receive accurate feedback about their environment.

Once the data is filtered, it is translated into sounds made by the buzzer. The buzzer's beeping interval varies based on the distance of obstacles and decreases as the objects come closer. This approach provides users with a real-time auditory map of their surroundings, enabling them to walk with more confidence.

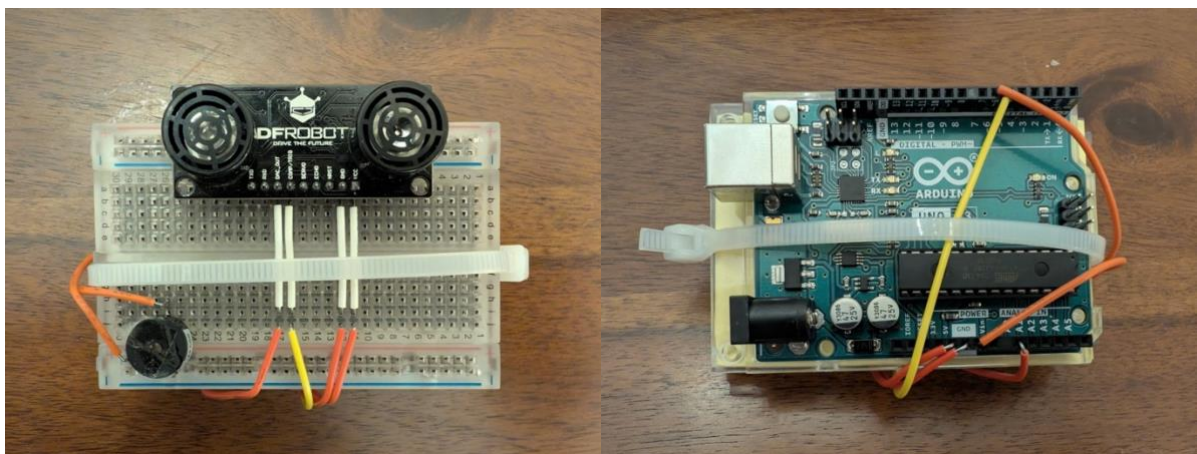


Figure 1: Project Hardware Setup Front (Left) & Back (Right)

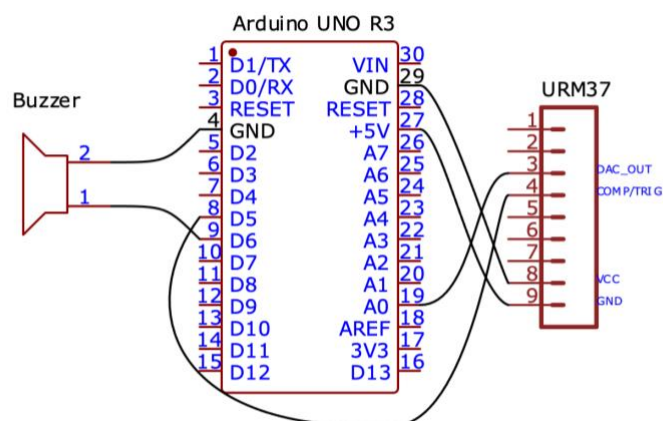


Figure 2: Circuit Schematic of the Connections

Methodology

To transform raw ultrasonic distance measurements into clear, actionable feedback for a visually impaired user, this project followed a structured approach combining hardware and software. The hardware setup included an ultrasonic sensor and a buzzer interfaced with an Arduino, while the software implemented noise filtering and feedback mechanisms. Together, they ensure accurate distance detection and meaningful alerts, enhancing user navigation.

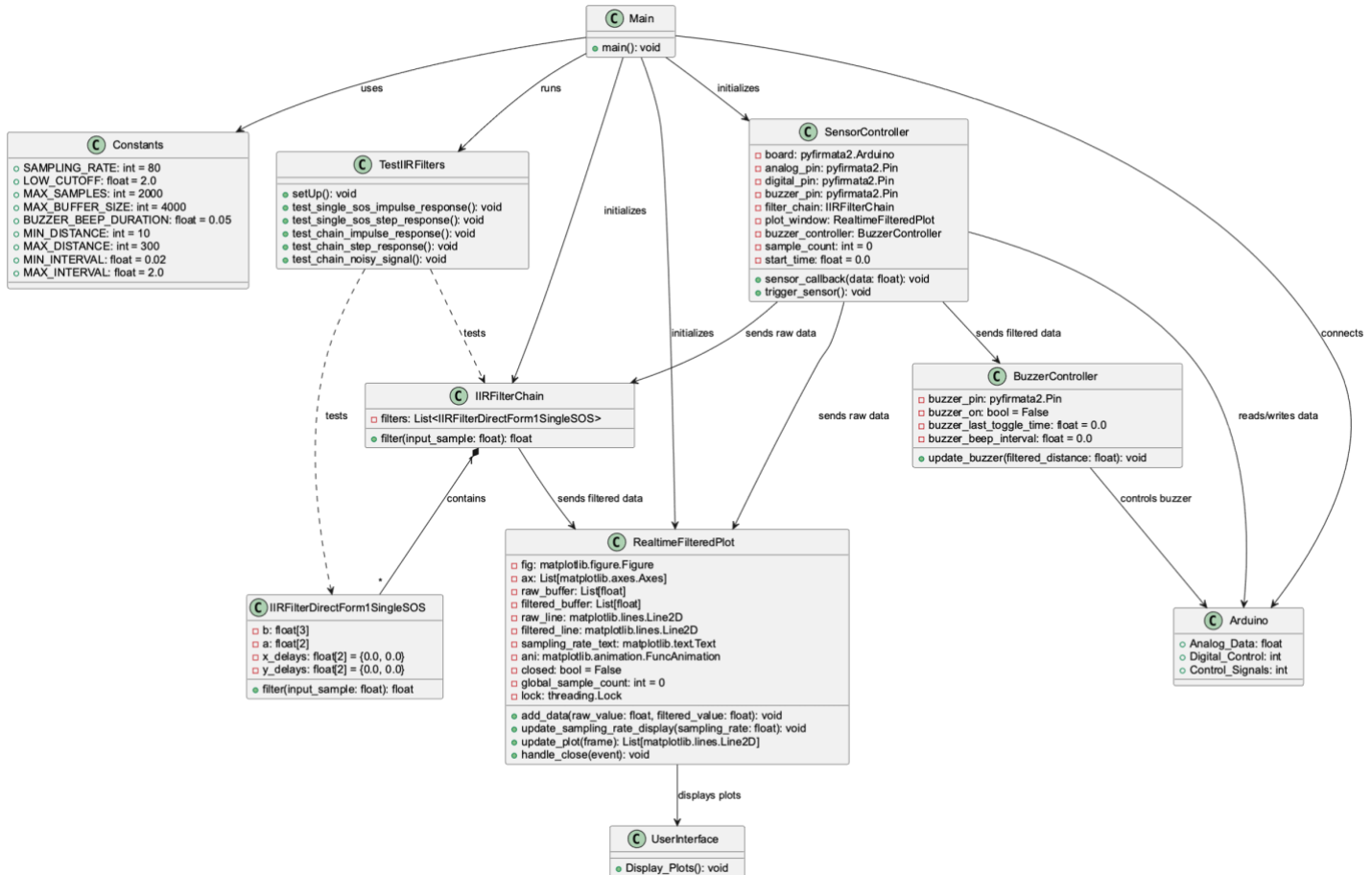


Figure 3: UML Dataflow Diagram

Sensor Integration and Data Acquisition:

The sensing framework utilizes a URM37 ultrasonic sensor connected to an Arduino microcontroller. The sensor's analogue voltage output, which is proportional to the detected distance. This is read through Arduino's analogue input pin A0. PyFirmata2 manages asynchronous data acquisition by registering the `sensor_callback` function to this analogue pin. When the Arduino transmits analogue data, PyFirmata2 invokes `sensor_callback`, which converts the raw ADC value to a distance measurement. This architecture ensures real-time processing of each incoming sample. Although PyFirmata2 supports sampling rates up to 1kHz, the system operates at 80Hz to balance responsiveness with computational efficiency.

Pre-Processing Steps and Sampling Rate Monitoring:

The application continuously monitored the sampling rate by tracking the number of samples acquired within fixed time intervals. This monitoring enabled the detection of deviations from the target sampling frequency of 80Hz. Maintaining a consistent sampling rate was essential to prevent jitter and timing discrepancies, which could compromise the quality of the filtered output.

Signal Filtering Using an IIR Filter Chain:

The implementation focuses on a real-time Infinite Impulse Response (IIR) filter implemented as a chain of second-order sections (SOS). The design process focused on selecting filter parameters that effectively reduce measurement noise while preserving the integrity of distance data. Utilizing SciPy, SOS coefficients were generated from a low-pass Butterworth filter.

This custom filter chain employed a direct form I architecture, where each incoming sample is sequentially processed through multiple second-order filters. Each stage progressively smooths the signal, eliminating sporadic spikes and rapid fluctuations. The resulting output provides stable, low-noise distance measurements. The filtering pipeline was validated through unit tests, which compared the custom implementation's responses to impulse, step, and noisy input signals against those produced by established reference functions like SciPy's `sosfilt`. These tests confirmed that the filter chain performs reliably across various test signals, ensuring its effectiveness with real-time data.

Feedback Generation Through a Buzzer Interface:

Filtered distance measurements are converted into auditory cues using a buzzer. A control algorithm dynamically adjusts the buzzer's beep interval based on the object's proximity: closer objects result in shorter intervals, signalling increased urgency, while farther objects produce longer intervals, allowing users to intuitively gauge distance changes. This adaptive feedback mechanism is managed by the `BuzzerController` class, ensuring that audio signals remain responsive and easily configurable to reflect real-time distance variations.

Data Visualization and Real-Time Assessment:

Along with the filtering and feedback routines, the system continuously plotted both raw and filtered distance measurements. By displaying the data streams side by side, one could visually confirm that the filter chain was performing as intended—stabilizing the input signal and closely tracking moving targets without overshooting or lagging. This immediate visualization also assisted in fine-tuning parameters, verifying that the chosen cutoff frequency and filter order were well suited for the application.

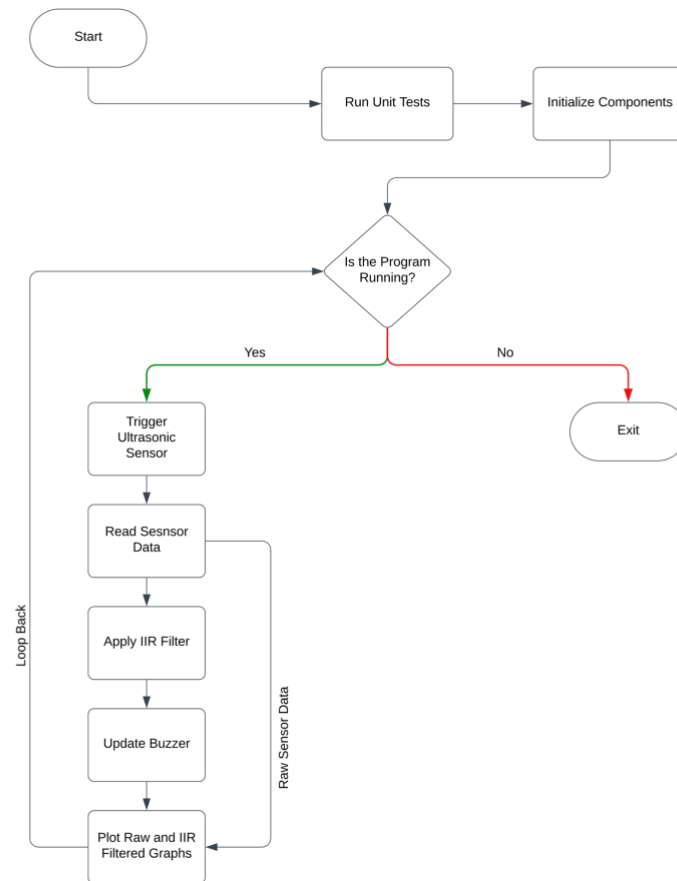


Figure 4: Flowchart of the Program

IIR Filtering

In real-time sensing applications, raw measurements often exhibit unwanted fluctuations, high-frequency noise, and transient artifacts that obscure the underlying signal of interest. For this project, the goal was to ensure that the ultrasonic sensor's distance readings—used to alert visually impaired users—were stable and reliable. Applying an Infinite Impulse Response (IIR) filter facilitated the production of a smooth, noise-reduced signal, enabling accurate auditory alerts through the buzzer.

Choosing a Low-Pass Filter:

The primary source of meaningful information in this scenario is the gradual change in distance as objects approach or recede. Sudden spikes or rapid jitter in the sensor's output are typically due to electrical noise, small vibrations, or environmental disturbances. Additionally, the continuous operation of the buzzer itself can introduce subtle mechanical vibrations or electrical interference that might affect nearby sensor readings. To address these issues, a low-pass filter was employed. This type of filter attenuates fast changes in the signal occurring at higher frequencies while allowing the slower, more relevant variations to pass.

The cutoff frequency was chosen as 2Hz. This decision was made from balancing responsiveness against smoothness. With the sensor sampling at 80Hz, the Nyquist frequency (half the sampling rate) is 40Hz. This setting ensures that rapid fluctuations from noise, including any unintended vibrational coupling from the buzzer's harmonics, are suppressed, allowing for stable readings. In practical terms, a person or obstacle in front of the sensor does

not move at dozens of cycles per second. Thus, 2Hz provides a sensible margin to capture realistic motion and changes in environment while filtering out jitters.

Determining the Filter Response and Generating SOS Coefficients:

To implement the IIR filter, a 4th-order low-pass Butterworth design was selected. A Butterworth filter offers a maximally flat frequency response in the passband, ensuring minimal ripple and a predictable behaviour that is straightforward to tune. Using Python's built-in `scipy.signal.butter` function, we computed the second-order section (SOS) coefficients, which were then utilized in the custom IIR filter class to preserve the low-frequency content of the ultrasonic measurements.

Real-Time IIR Filter Class as a Chain of Direct Form I Filters:

To meet the assignment's requirement of writing a real-time IIR filter class without relying on high-level filtering functions during execution, a chain of second-order section coefficients in Direct Form I were constructed. Each stage of the filter is implemented as a separate class instance that handles two input delays, two output delays, and a single accumulator. By connecting multiple second-order filters end-to-end, we achieve the desired 4th-order response.

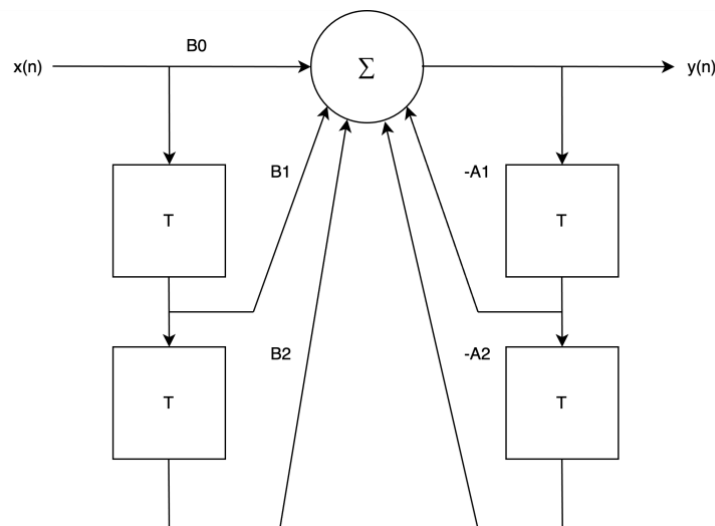


Figure 5: Direct Form I Filter with One Accumulator

Unit Testing and Verification:

To ensure correctness, a suite of unit tests was implemented. These tests verified that the custom IIR filter's impulse and step responses, as well as its performance on noisy signals, matched the known reference outputs produced by `sosfilt`. By testing with known inputs, we confirmed that the chain of filters was accurately realized and stable, making it suitable for Realtime applications. If the tests failed, the code was designed to exit before real-time processing was started.

Interpreting the Graphs:

Three key plots were generated to illustrate the filter's effectiveness:

1. Normalized Raw Frequency Response of the Ultrasonic Sensor:

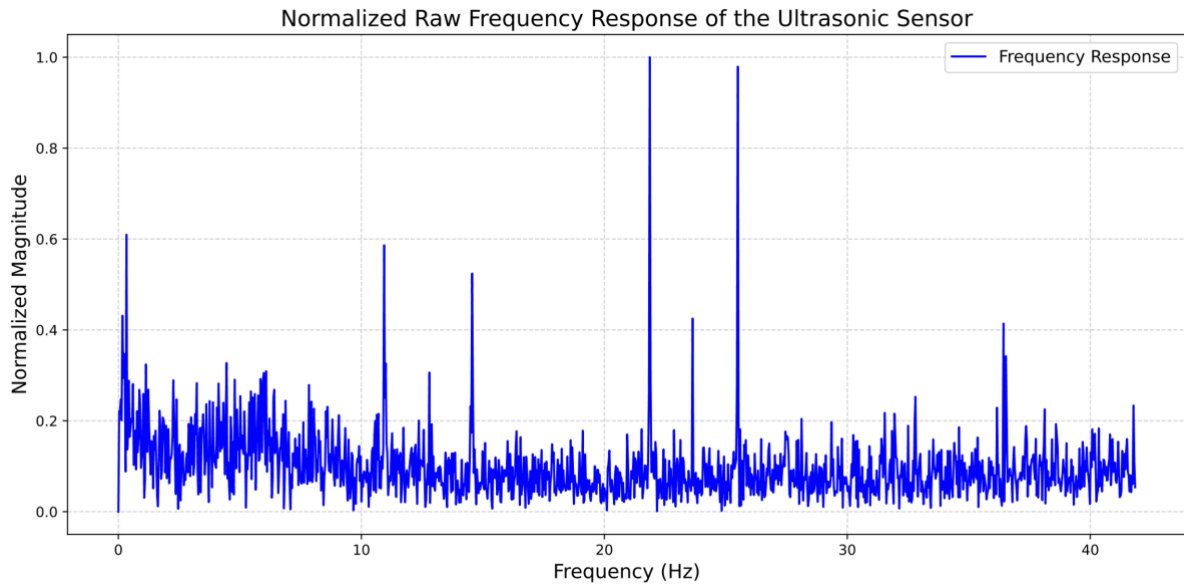


Figure 6: Raw Data Frequency Response

This plot shows the frequency spectrum of the unfiltered signal. Peaks at higher frequencies indicate unwanted noise and rapid oscillations that make accurate interpretation of the distance data challenging.

2. Normalized Filtered Frequency Response of the Ultrasonic Sensor:

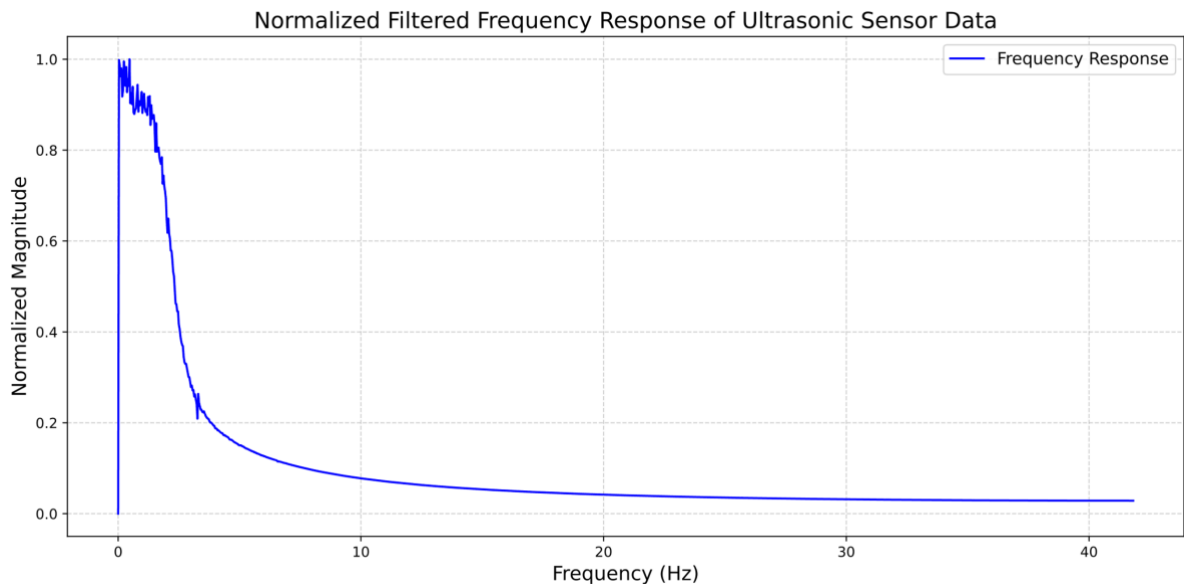


Figure 7: Filtered Data Frequency Response

After applying the low-pass IIR filter, the resulting frequency response reveals a significant attenuation of frequencies above 2Hz. The filtered spectrum appears more concentrated near zero frequency, reflecting a smoother, more stable signal that better represents everyday variations in distance. In addition to that, any high-frequency noise from the buzzer's harmonics or any other environmental factors is mitigated.

3. Time Domain Plots of Raw vs. IIR Filtered Ultrasonic Data:

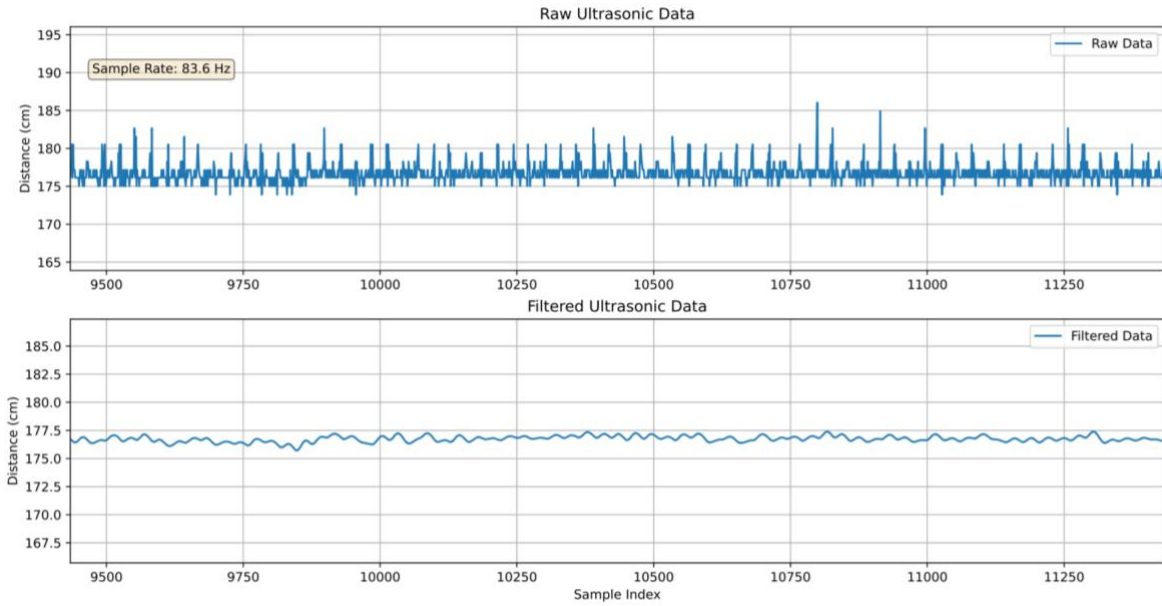


Figure 8: Real-Time Raw vs. IIR Filtered Ultrasonic Data

Before filtering, the plot of the distance readings shows jitter, sudden spikes, and rapid fluctuations. These high-frequency components, and harmonic noise from the buzzer interfering with the ultrasonic sensor's data, can lead to inaccurate buzzer triggers, reducing the reliability of auditory feedback. The IIR filtered plot illustrates the smoothed signal, where slow, meaningful trends in object proximity become prominent, and erratic changes are greatly reduced. By ensuring only gradual changes pass through, the buzzer's feedback helps the user distinguish genuine proximity alerts from random bursts of noise.

Sampling Rate of Acquisition

Monitoring the sampling rate during runtime is crucial for ensuring that ultrasonic sensor data remains accurate and reliable. This project employs a method of continuously measuring how many samples arrive over a known time interval. By tracking the start of data collection and the number of samples received at any given moment in a time interval, it is possible to compute how frequently the system captures new information. Regularly recording the time difference between the arrival of these samples allows the system to derive the current sampling frequency dynamically. This approach enables real-time detection of deviations in the intended sampling rate, which could occur due to computational overhead tasks, such as data filtering, plotting, or other simultaneous operations.

While the intended sampling rate for this project is 80 Hz, observations reveal that the actual rate varies slightly above this value, approximately at 83 Hz. This discrepancy arises due to the behaviour of the `board.samplingOn()` method in `pyfirmata2`, which determines the sampling interval in integer milliseconds. The calculation for the sampling interval is as follows:

Intended Sampling Rate: 80 Hz

Calculated Interval:

$$\frac{1000\text{ms}}{80\text{Hz}} = 12.5\text{ms}$$

However, the method's limitation to integer millisecond intervals introduces a rounding error:

- **Rounded Down to 12ms:**

$$\frac{1000}{12} \approx 83.33\text{Hz}$$

- **Rounded Up to 13ms:**

$$\frac{1000}{13} \approx 76.92\text{Hz}$$

As a result, setting `board.samplingOn(12)` results in a sampling rate of approximately 83Hz, which aligns with the observed value of being slightly higher than the intended 80Hz. It is also important to note that the live sample rate never fell below the chosen one. This ensured that expected data samples were not lost.

Significance of Sampling Rate Stability:

If the sampling rate deviates from the target frequency, the characteristics of the digital filters may change, potentially compromising the accuracy of the feedback mechanism. For instance, a lower sampling rate would affect the low-pass filter's behaviour, leading to altered timing and precision in the feedback provided to the user through the buzzer signals. Maintaining a consistent sampling rate ensures: the low-pass filter operates as designed and the feedback system remains reliable and timely.

Real-time monitoring of the sampling frequency also aids in identifying and diagnosing performance bottlenecks, enabling prompt adjustments to the code or system configuration. This ensures that the quality of the filtered output and the reliability of the buzzer signals, which users depend on, remain uncompromised.

Critical Analysis and Conclusion

The project successfully developed a reliable ultrasonic sensing system to assist visually impaired individuals by implementing a low-pass IIR filter that effectively reduced high-frequency noise in raw measurements. This enabled the buzzer's beeping pattern to accurately reflect obstacle positions, thereby enhancing user safety and confidence. However, challenges such as computational overhead from using Matplotlib for real-time plotting introduced minor discrepancies in data visualization. Transitioning to a more efficient plotting library like PyQtGraph could mitigate these issues, improving system responsiveness. Additionally, maintaining a consistent sampling rate is crucial; while the system reliably achieved or exceeded the target of 80Hz, further optimization in data handling and multithreading could enhance stability and reduce latency. Refinements in the buzzer feedback logic are also necessary to increase the intuitiveness and responsiveness of auditory cues, ensuring the system remains user-friendly.

Future iterations can build upon these findings by refining both the technology and its implementation. Integrating the system into a smaller, lightweight design, such as a pair of

smart glasses, would reduce weight and improve portability, offering hands-free operation while maintaining navigation functionality. Incorporating a haptic motor could eliminate noise interference from the buzzer's harmonics, enhancing the accuracy of ultrasonic sensor data without compromising feedback quality and preventing disturbance to others nearby. Comprehensive real-world testing is essential to validate the system's effectiveness in diverse environments, addressing human factors and varying obstacle types. By optimizing computational performance, ensuring sampling rate stability, and enhancing feedback mechanisms, the project can transition from a promising prototype to a fully polished and effective assistive device.

Appendix

main.py:

```
import pyfirmata2
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation
import time
from scipy.signal import butter, sosfilt
import unittest
import threading

# ===== Constants =====
SAMPLING_RATE = 80          # Sampling rate
LOW_CUTOFF = 2              # Low cutoff frequency
MAX_SAMPLES = 2000          # Max samples for plotting window
MAX_BUFFER_SIZE = 4000      # Max buffer size before trimming data
BUZZER_BEEP_DURATION = 0.05 # Buzzer beep duration
MIN_DISTANCE = 10           # Min distance to trigger buzzer
MAX_DISTANCE = 300          # Max distance for buzzer
MIN_INTERVAL = 0.02         # Min beep interval
MAX_INTERVAL = 2.0          # Max beep interval

# ===== Filter Classes =====
class IIRFilterDirectForm1SingleSOS:
    def __init__(self, sos_row):
        self.b = sos_row[:3]          # Numerator coeff.
        self.a = sos_row[3:]          # Denominator coeff.
        self.x_delays = np.zeros(2)   # 2 prev. input samples
        self.y_delays = np.zeros(2)   # 2 prev. output samples

    def filter(self, input_sample):    # Applies IIR to single sample
        output = (
            self.b[0] * input_sample +          # Direct form 1 equation
            self.b[1] * self.x_delays[0] +
            self.b[2] * self.x_delays[1] -
            self.a[1] * self.y_delays[0] -
            self.a[2] * self.y_delays[1]
        )
        self.x_delays[1] = self.x_delays[0]    # Update delay buffers
        self.x_delays[0] = input_sample
        self.y_delays[1] = self.y_delays[0]
        self.y_delays[0] = output
        return output

class IIRFilterChain:             # Processes input through a chain of second-order
    IIR filters
    def __init__(self, sos):
```

```

        self.filters = [IIRFilterDirectForm1SingleSOS(s) for s in sos] #
Initialise filters from sos coeff.

    def filter(self, input_sample):
        for f in self.filters:
            input_sample = f.filter(input_sample)        # Sequentially apply
each filter
        return input_sample

# ===== SOS Coefficients =====
sos = butter(4, LOW_CUTOFF / (SAMPLING_RATE / 2), btype='low',
output='sos')    # 4th order Butterworth lowpass

# ===== Unit Tests =====
class TestIIRFilters(unittest.TestCase):    # Validates test signals in
our iir implementation vs scipy's sosfilt.
    def setUp(self):
        self.sos = sos
        self.impulse = [1] + [0]*19    # Delta function
        self.step = [1]*20    # Step signal
        t = np.arange(0, 1, 1/SAMPLING_RATE) # Time vector
        np.random.seed(0)    # Seed for fixed starting point
        self.noisy_signal = np.sin(2*np.pi*20*t) + 0.5*np.random.randn(len(t))
# 20Hz sin wave with noise

    def test_single_sos_impulse_response(self):
        single_sos_filter = IIRFilterDirectForm1SingleSOS(self.sos[0])
        output = [single_sos_filter.filter(x) for x in self.impulse]
        expected_output = sosfilt([self.sos[0]], self.impulse)
        np.testing.assert_almost_equal(output, expected_output, decimal=5)

    def test_single_sos_step_response(self):
        single_sos_filter = IIRFilterDirectForm1SingleSOS(self.sos[0])
        output = [single_sos_filter.filter(x) for x in self.step]
        expected_output = sosfilt([self.sos[0]], self.step)
        np.testing.assert_almost_equal(output, expected_output, decimal=5)

    def test_chain_impulse_response(self):
        chain_filter = IIRFilterChain(self.sos)
        output = [chain_filter.filter(x) for x in self.impulse]
        expected_output = sosfilt(self.sos, self.impulse)
        np.testing.assert_almost_equal(output, expected_output, decimal=5)

    def test_chain_step_response(self):
        chain_filter = IIRFilterChain(self.sos)
        output = [chain_filter.filter(x) for x in self.step]
        expected_output = sosfilt(self.sos, self.step)
        np.testing.assert_almost_equal(output, expected_output, decimal=5)

```

```

def test_chain_noisy_signal(self):
    chain_filter = IIRFilterChain(self.sos)
    output = [chain_filter.filter(x) for x in self.noisy_signal]
    expected_output = sosfilt(self.sos, self.noisy_signal)
    np.testing.assert_almost_equal(output, expected_output, decimal=5)

# ===== Buzzer Controller =====
class BuzzerController:
    # Controls the buzzer based on filtered
    # distance measurements.
    def __init__(self, buzzer_pin):
        self.buzzer_pin = buzzer_pin
        self.buzzer_on = False
        self.buzzer_last_toggle_time = time.perf_counter()
        self.buzzer_beep_interval = 0

    def update_buzzer(self, filtered_distance):
        # Compute beep
        # interval based on distance
        current_time = time.perf_counter()
        if filtered_distance <= MIN_DISTANCE:
            self.buzzer_beep_interval = MIN_INTERVAL
            # Close = fast beeps
        elif filtered_distance >= MAX_DISTANCE:
            self.buzzer_beep_interval = MAX_INTERVAL
            # Far = slow beeps
        else:
            # Logarithmical interval based on min max distance
            normalized_distance = (filtered_distance - MIN_DISTANCE) /
            (MAX_DISTANCE - MIN_DISTANCE)
            k = np.log(MAX_INTERVAL / MIN_INTERVAL)
            beep_interval = MIN_INTERVAL * np.exp(k * normalized_distance)
            beep_interval = max(MIN_INTERVAL, min(beep_interval,
            MAX_INTERVAL))
            self.buzzer_beep_interval = beep_interval
        if self.buzzer_on:
            # Toggle buzzer state
            if current_time - self.buzzer_last_toggle_time >=
            BUZZER_BEEP_DURATION:
                self.buzzer_pin.write(0)
                self.buzzer_on = False
                self.buzzer_last_toggle_time = current_time
        else:
            if current_time - self.buzzer_last_toggle_time >=
            self.buzzer_beep_interval:
                self.buzzer_pin.write(1)
                self.buzzer_on = True
                self.buzzer_last_toggle_time = current_time

# ===== Real-Time Plot Class =====
class RealtimeFilteredPlot:
    # Realtime raw and filtered data plots
    def __init__(self, max_samples=MAX_SAMPLES):

```

```

        self.fig, self.ax = plt.subplots(2, 1, figsize=(12, 8))      # 2 plots:
raw and filtered
        self.max_samples = max_samples
        self.raw_buffer = []
        self.filtered_buffer = []
        self.raw_line, = self.ax[0].plot([], [], label='Raw Data')
        self.filtered_line, = self.ax[1].plot([], [], label='Filtered Data')
        # Titles and labels
        self.ax[0].set_title('Raw Ultrasonic Data')
        self.ax[1].set_title('Filtered Ultrasonic Data')
        self.ax[1].set_xlabel('Sample Index')
        self.ax[0].set_ylabel('Distance (cm)')
        self.ax[1].set_ylabel('Distance (cm)')
        self.ax[0].grid(True)      # Finer gridlines on plot
        self.ax[1].grid(True)
        self.ax[0].legend()
        self.ax[1].legend()

        self.sampling_rate_text = self.ax[0].text(      #live samplerate on
plot
            0.02, 0.92, 'Sample Rate: 0.0 Hz', transform=self.ax[0].transAxes,
            fontsize=10, verticalalignment='top',
            bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5)
        )
        self.ani = animation.FuncAnimation(      #initialise plt animation
            self.fig, self.update_plot, interval=50, cache_frame_data=False
        )
        self.closed = False      # Flag
        self.fig.canvas.mpl_connect('close_event', self.handle_close)

        self.global_sample_count = 0      # Initialize sample counter
        self.lock = threading.Lock()      # Lock threads

    def handle_close(self, event):      # When window is closed
        print('Figure closed')
        self.closed = True
        if self.ani.event_source is not None:      # Stop the animation to
prevent further updates
            self.ani.event_source.stop()
        plt.close(self.fig)

    def update_plot(self, frame):      # Updates the plot with new data
        with self.lock:
            buffer_length_raw = len(self.raw_buffer)
            buffer_length_filtered = len(self.filtered_buffer)

            if buffer_length_raw != buffer_length_filtered:      # Ensure
both buffers have the same length for synced plotting

```

```

        min_length = min(buffer_length_raw, buffer_length_filtered)
        print(f"Buffer length mismatch: raw={buffer_length_raw},
filtered={buffer_length_filtered}. Trimming to {min_length}")
        raw_data = self.raw_buffer[-min_length:]
        filtered_data = self.filtered_buffer[-min_length:]
    else:
        min_length = buffer_length_raw
        raw_data = self.raw_buffer
        filtered_data = self.filtered_buffer

    if min_length > 0:
        x_start = self.global_sample_count - min_length      # x-axis
based on global_sample_count
        x_end = self.global_sample_count
        x_data = np.arange(x_start, x_end)

        if min_length > self.max_samples:                    # Limit data to
max no. of samples
            x_data = x_data[-self.max_samples:]
            raw_data = raw_data[-self.max_samples:]
            filtered_data = filtered_data[-self.max_samples:]

        self.raw_line.set_data(x_data, raw_data)              # Update plot
lines with new data
        self.filtered_line.set_data(x_data, filtered_data)

        if len(x_data) > 0:
            if len(x_data) > self.max_samples:
                self.ax[0].set_xlim(x_data[0], x_data[-1])    #
Adjust x-axis limits based on data
                self.ax[1].set_xlim(x_data[0], x_data[-1])
            else:
                self.ax[0].set_xlim(max(0, self.global_sample_count -
self.max_samples), self.global_sample_count)
                self.ax[1].set_xlim(max(0, self.global_sample_count -
self.max_samples), self.global_sample_count)

        self.ax[0].set_ylim(                                  # Update Y-limits based on
data
            min(raw_data) - 10,
            max(raw_data) + 10
        )
        self.ax[1].set_ylim(
            min(filtered_data) - 10,
            max(filtered_data) + 10
        )
    else:
        # If no data yet, set default limits
        self.ax[0].set_xlim(0, self.max_samples)

```

```

        self.ax[1].set_xlim(0, self.max_samples)

    return self.raw_line, self.filtered_line, self.sampling_rate_text

def add_data(self, raw_value, filtered_value):      # Adds new data to
buffers
    with self.lock:
        self.raw_buffer.append(raw_value)
        self.filtered_buffer.append(filtered_value)
        self.global_sample_count += 1 # Increment sample counter

        if len(self.raw_buffer) > MAX_BUFFER_SIZE:
            trimmed = MAX_BUFFER_SIZE // 2
            del self.raw_buffer[:trimmed]          # Trims buffer to manage
ram consumption
            del self.filtered_buffer[:trimmed]

    def update_sampling_rate_display(self, sampling_rate): # Updates sampling
rate on plot
        with self.lock:
            self.sampling_rate_text.set_text(f'Sample Rate:
{sampling_rate:.1f} Hz')

# ===== Sensor Controller =====
class SensorController:      # Manages the Arduino
    def __init__(self, board, analog_pin, digital_pin, buzzer_pin,
filter_chain, plot_window):
        self.board = board
        self.analog_pin = analog_pin
        self.digital_pin = digital_pin
        self.buzzer_pin = buzzer_pin
        self.filter_chain = filter_chain
        self.plot_window = plot_window
        self.buzzer_controller = BuzzerController(self.buzzer_pin)

    # Setup sensor and buzzer
    self.digital_pin.write(1) # Trigger ultrasonic sensor
    self.buzzer_pin.write(0) # Ensure buzzer is off
    self.analog_pin.register_callback(self.sensor_callback)
    self.analog_pin.enable_reporting() # Start data stream

    self.sample_count = 0 # Initialise Fs calculation
    self.start_time = time.perf_counter() # Time stamp

    def sensor_callback(self, data):      # Triggred by analogue pin data
        if data is not None:
            adc_value = data * 1023.0 # 10 bit ADC (L-1)

```



```

        distance_measured = ((adc_value * 1.1) - 100.0)          #
(SensorValue * 5000 / 1024 ) / 4.125 - 100
        filtered_distance =
self.filter_chain.filter(distance_measured)      # Apply IIR
        self.plot_window.add_data(distance_measured,
filtered_distance)      # Add new data to window
        self.buzzer_controller.update_buzzer(filtered_distance) # Update
buzzer based on dist.

        # Update sampling rate
        self.sample_count += 1
        current_time = time.perf_counter()
        elapsed_time = current_time - self.start_time
        if elapsed_time >= 1.0:
            sampling_rate = self.sample_count / elapsed_time      #
calculate samples/sec for Fs
            self.plot_window.update_sampling_rate_display(sampling_rate)
        # Update plot display
            self.sample_count = 0          # Reset counter
            self.start_time = current_time

def trigger_sensor(self):          # 1/0 to D5 COMP/TRIG
    self.digital_pin.write(0)
    time.sleep(0.01)
    self.digital_pin.write(1)

# ===== Main Function =====
def main():
    # Start Tests
    print("Running unit tests...")
    test_loader = unittest.TestLoader()
    test_suite = test_loader.loadTestsFromTestCase(TestIIRFilters)
    test_runner = unittest.TextTestRunner(verbosity=2)
    test_results = test_runner.run(test_suite)
    if not test_results.wasSuccessful():
        print("Unit tests failed. Exiting the program.")
        return

    print("Unit tests passed. Starting real-time processing...")

    PORT = pyfirmata2.Arduino.AUTODETECT      # Autodetect arduino
    board = None
    try:
        board = pyfirmata2.Arduino(PORT)
        print("Connected to Arduino.")
    except Exception as e:
        print(f"Error: Unable to connect to board. {e}")
    return

```

```

# Define pins on Arduino
analog_pin = board.get_pin('a:0:i')
digital_pin = board.get_pin('d:5:o')
buzzer_pin = board.get_pin('d:6:o')
# Initialisation
filter_chain = IIRFilterChain(sos)
plot_window = RealtimeFilteredPlot()
sensor_controller = SensorController(board, analog_pin, digital_pin,
buzzer_pin, filter_chain, plot_window)

try:
    board.samplingOn(1000 / SAMPLING_RATE) # Sampling interval in ms
    print(f"Sampling started at {SAMPLING_RATE} Hz.")
    while not plot_window.closed:
        sensor_controller.trigger_sensor() # trigger D5
        time.sleep(1 / SAMPLING_RATE)    # Wait for next sampling
interval
        plt.pause(0.01)                  # Allow plt to process

except KeyboardInterrupt:
    print("Exiting program.")            # ctrl + c
except Exception as e:
    print(f"An unexpected error occurred: {e}")
finally:
    if board is not None:
        try:
            buzzer_pin.write(0)          # Ensure buzzer is turned off
            board.exit()                  # Closes connection to arduino
            print("Arduino connection closed.")
        except AttributeError:
            print("Board was not fully initialized. Skipping exit.")
        except Exception as e:
            print(f"Error during board exit: {e}")

if __name__ == "__main__":
    main()

```

Unit Test Proof (Snapshot of Terminal):

```

PS C:\Users\Agnisys84\Documents\DSP\assignment_3> & C:/Users/Agnisys84/AppData/Local/Programs/Python/Python312/python.exe c:/Users/Agnisys84/Documents/DSP/assignment_3/main.py
Running unit tests...
test_chain_impulse_response (__main__.TestIIRFilters.test_chain_impulse_response) ... ok
test_chain_noisy_signal (__main__.TestIIRFilters.test_chain_noisy_signal) ... ok
test_chain_step_response (__main__.TestIIRFilters.test_chain_step_response) ... ok
test_single_sos_impulse_response (__main__.TestIIRFilters.test_single_sos_impulse_response) ... ok
test_single_sos_step_response (__main__.TestIIRFilters.test_single_sos_step_response) ... ok

-----
Ran 5 tests in 0.002s

OK
Unit tests passed. Starting real-time processing...
Connected to Arduino.
Sampling started at 80 Hz.
Figure closed
Arduino connection closed.
PS C:\Users\Agnisys84\Documents\DSP\assignment_3> 

```